

スクリプトに落とせない反復作業を Claude に覚えさせる ～ Skills 入門 ～

Myxogastria0808

1. 目次

1. 目次

1. 目次	1
2. Skills とは？	3
2.1. 概要	4
2.2. 利用場面	5
3. Skills の作り方	6
3.1. 概要	7
3.2. はじめに	8
3.3. ① SKILL.md を作成する	9
3.4. ② SKILL.md の内容を記述する	11
3.5. ③ 作成した Skills の挙動を確認する	20
4. おわりに	21
5. 参考文献	23

2. Skills とは？

2.1. 概要

Claude の機能を拡張するための仕組み。Skills を使うことで、Claude に対して繰り返し依頼する作業を、少ない労力で、より簡単に行うことができる。

特に、スクリプトに落とし込めない曖昧な処理を含む作業に対して、Skills は有効である。

※ 注意事項

このスライド資料は、Claude Code Docs (公式ドキュメント)を元に、biopackathon 向けに再構成したものである。それぞれの情報の詳細については、公式ドキュメントを参照のこと。

2.2. 利用場面

- 自然言語を書かせる作業の例
 - ▶ ソースコードの変更や追加に合わせて、コメントアウトの修正を行う作業を自動化したい場合
 - ▶ 日報の内容について、箇条書きの情報を与えて、Claude に文章化してもらう作業を自動化したい場合
 - ▶ 自然言語に関する修正について、文字列置換で修正できないような類似した内容の修正を自動化したい場合
- Claude のハーネスに関する例
 - ▶ Claude に対して、使わせたいツールがある場合に、できる限り強制的に使わせたい場合
 - ▶ おおよそ同じプロンプトを繰り返し使う作業を自動化したい場合

上記の例以外にも、Claude に対して繰り返し依頼する作業を登録できる。

3. Skills の作り方

3.1. 概要

簡単な Python スクリプトと docstring が書かれていて、これらの情報を元に Sphinx がドキュメントを生成するプロジェクトを例に、Skills の作り方を説明する。

対象のレポジトリは、以下の URL からクローンできる。

<https://github.com/Myxogastria0808/biopackathon-claude-skills>

docstring のスタイルは、Google Style や Numpy Style など複数のスタイルが存在する。Python スクリプトの変更や追加がある度に、docstring の内容を更新・追加する必要がある。しかし、docstring のスタイルを統一しつつ、丁寧にコメントを記述していくことは非常に大変である。

Skills を作成することにより、docstring の内容を Python スクリプトに追従させる作業負担を大幅に軽減することができる。今回は、このような Skills の作り方を例に説明する。

3.2. はじめに

これから示す Skills の作成手順は、あくまで一例であることに留意していただきたい。より詳細な情報を知りたい場合は、以下の公式ドキュメントを参照していただきたい。

- Claude Code Docs: Skills > スキルで Claude を拡張する

<https://code.claude.com/docs/ja/skills>

- Claude Code Docs: プラグイン > プラグインを作成する

<https://code.claude.com/docs/ja/plugins>

- Claude Code Docs: プラグイン > マーケットプレイスから事前構築されたプラグインを発見してインストールする

<https://code.claude.com/docs/ja/discover-plugins>

3.3. ① SKILL.md を作成する

Skills のエントリポイントは SKILL.md というファイルであり、このファイルを置いたディレクトリがそのままひとつの Skills になる。

保存する場所によって、誰がその Skills を使えるかが変わる。

- Personal: ~/.claude/skills/<skill-name>/SKILL.md
→ 全プロジェクトで利用可能
- Project: .claude/skills/<skill-name>/SKILL.md
→ そのプロジェクトのみで利用可能
(バージョン管理に含めれば、チームで共有できる)

また、ディレクトリ名がそのまま Skills の呼び出しコマンド名になる (.claude/skills/update-docstring/ なら /update-docstring)。

今回は docstring を追従させる作業をプロジェクトのメンバー全員で共有したいため、Project スキルとして作成する。

3. Skills の作り方

従って、Skills は以下のように作成すれば良い。

```
mkdir -p .claude/skills/update-docstring  
touch .claude/skills/update-docstring/SKILL.md
```

上記の例からもわかる通り、`.claude/skills/<skill-name>/`ディレクトリの下に、`SKILL.md`を作成するだけで、Skills の作成は完了する。

3.4. ② SKILL.md の内容を記述する

SKILL.md は、YAML フロントマターフィールド (---で挟まれた部分) と、 それに続く Markdown コンテンツの 2 つから構成される。

YAML フロントマターフィールド

YAML フロントマターフィールドには、公式ドキュメントに記載されているフィールドを必要に応じて記述する。(公式ドキュメントには、フロントマターリファレンスとして、利用可能なフィールドの一覧が記載されている。)

Markdown コンテンツ

Markdown コンテンツには、任意の内容を記述できるが、推奨される書き方が存在する。

3.4. ② SKILL.md の内容を記述する

YAML フロントマターフィールド

YAML フロントマターフィールドは、全て任意のフィールドであり、必要に応じて記述する。

以下に、いくつか便利なフィールドを紹介する。

name

スキルリストに表示される表名。(default: ディレクトリ名)

description

スキルの概要と使用タイミングを書くのが望ましい。(default: Markdown コンテンツの最初の段落)

3.4. ② SKILL.md の内容を記述する

disable-model-invocation

true にすると、Claude が自動でスキルを呼び出さなくなる。その場合、ユーザーが明示的に /skill-name で呼び出す必要がある。(default: false)

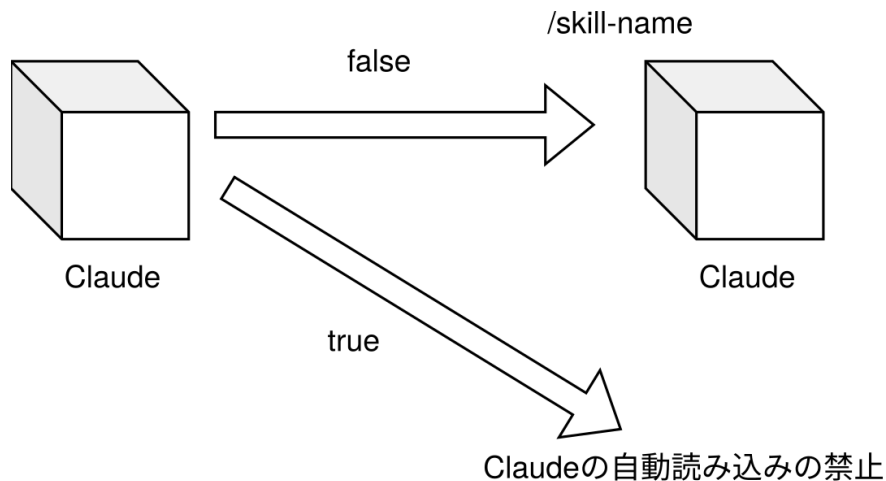


Figure 1: disable-model-invocation

3.4. ② SKILL.md の内容を記述する

user-invokable

false にすると、ユーザーが /skill-name で呼び出すことができなくなる。(default: true)

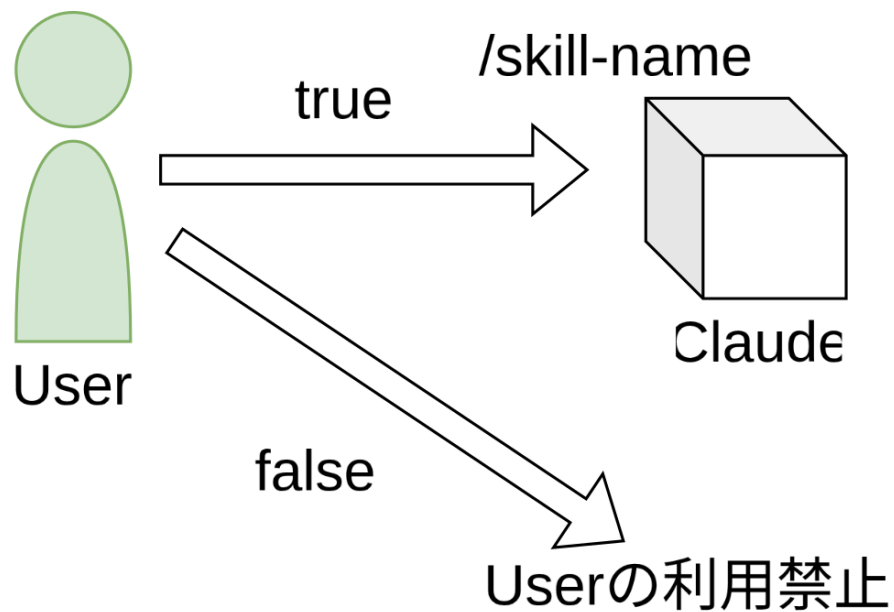


Figure 2: user-invokable

3.4. ② SKILL.md の内容を記述する

arguments

Skills には、引数を渡すことができる。ユーザから渡された引数は、\$ARGUMENTS[N] という形で 0 ベースのインデックスで特定の引数にアクセス可能である。しかし、引数の数が多くなると、どの引数がどのインデックスに対応しているかがわかりにくくなる。そのような場合に、arguments フィールドを使って、引数の名前を与えることができる。

argument-hint

ユーザが使用する Skills に与えるべき引数がわかるように、引数のヒントを与えることができる。

arguments と argument-hint を用いたサンプルとして、/argument-sample という Skills を用意している。

3. Skills の作り方

/argument-sample の SKILL.md の内容は以下の通りである。

```
---
name: argument-sample
description: Demonstrates the skill arguments feature — $ARGUMENTS, $ARGUMENTS[N] / $N, and named arguments
declared via the arguments frontmatter field
disable-model-invocation: true
argument-hint: [first] [second]
arguments: [first, second]
---
```

1. Print "step-1 all arguments (\\$ARGUMENTS): \$ARGUMENTS"
 2. Print "step-2 first positional argument (\\$ARGUMENTS[0] / \\$0): \$0"
 3. Print "step-3 second positional argument (\\$ARGUMENTS[1] / \\$1): \$1"
 4. Print "step-4 named argument \\$first (declared via `arguments: [first, second]` in the frontmatter): \$first"
 5. Print "step-5 named argument \\$second (declared via `arguments: [first, second]` in the frontmatter): \$second"
-

3.4. ② SKILL.md の内容を記述する

argument-sample の Skills を呼び出すと、以下のように引数を渡すことができる。

```
> /argument-sample [first] [second]
```

Figure 3: argument-sample

実行結果は以下の通りである。

```
> /argument-sample apple banana
● step-1 all arguments ($ARGUMENTS): apple banana
  step-2 first positional argument ($ARGUMENTS[0] / $0): apple
  step-3 second positional argument ($ARGUMENTS[1] / $1): banana
  step-4 named argument $first (declared via arguments: [first, second] in the frontmatter): apple
  step-5 named argument $second (declared via arguments: [first, second] in the frontmatter): banana
* Cogitated for 2s
```

Figure 4: argument-sample の実行結果

3.4. ② SKILL.md の内容を記述する

Markdown コンテンツ

Markdown コンテンツには、任意の内容を記述できるが、次のような推奨される書き方が存在する。

SKILL.md 自体は簡潔に保ち、500 行を超えるような詳細なリファレンスは別ファイルに切り出して SKILL.md から参照するのが推奨されている。Markdown 中に、`[reference.md](reference.md)` のように記述することで、別ファイルに切り出した詳細なリファレンスの内容と読み込みタイミングを Claude に伝えられる。

次に、Markdown コンテンツで利用可能なパターンを紹介する。

3.4. ② SKILL.md の内容を記述する

動的コンテキストの注入

インラインフォーム `!`<command>``、または、コードブロック ````!<command>```` 構文は、Skills の内容が Claude に渡される前に、指定されたシェルコマンドを実行し、その出力を構文に対して置き換えることができる。

イメージ図

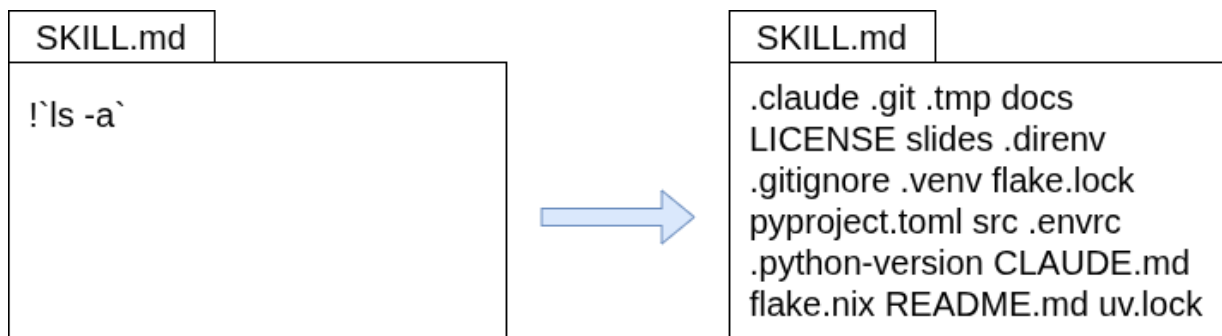


Figure 5: argument-sample の実行結果

3.5. ③ 作成した Skills の挙動を確認する

Skills が意図通りに働くかは、Claude が呼び出してくれるかどうかと、実行結果が期待通りかどうかを別々に確認する必要がある。

確認方法は 2 通りある。

- description に合致しそうな頼み方をして、Claude に自動で呼び出させる
例：「lib.py の docstring を最新化して」
- コマンド名で明示的に呼び出す
例：/update-docstring

lib.py に関数を追加して docstring を書かない（あるいは古いまま残す）状態を作り、上記のいずれかの方法で Claude に依頼すると、update-docstring の Skills が読み込まれ、手順に沿って docstring が更新される。

4. おわりに

ここまで、Skills の作り方について簡単に説明してきた。
しかし、これらの内容を人間が全て手作業が行う必要は全くない。Skills の作成そのものも、Claude に依頼することで大幅に作業負担を軽減することができる。
Skills の仕組みの概要を知っていれば、Claude に生成させた Skills の調整も容易になることだろう。

よい Claude ライフを！

5. 参考文献

5. 参考文献

- Claude Code Docs: Skills > スキルで Claude を拡張する

<https://code.claude.com/docs/ja/skills>

- Claude Code Docs: プラグイン > プラグインを作成する

<https://code.claude.com/docs/ja/plugins>

- Claude Code Docs: プラグイン > マーケットプレイスから事前構築されたプラグインを発見してインストールする

<https://code.claude.com/docs/ja/discover-plugins>

- uv の使い方

<https://qiita.com/futakuchi0117/items/9ec8bd84797fed180647>

5. 参考文献

- Sphinx の使い方

<https://qiita.com/futakuchi0117/items/4d3997c1ca1323259844>

<https://www.sphinx-doc.org/en/master/usage/extensions/index.html>

<https://zenn.dev/yamagishihrd/articles/8b5279a07000a4>
